ORIGINAL ARTICLE

Enhanced sentiment classification of phone brands on Twitter with a modified walrus optimizer and novel ensemble method

Gomaa Elmokhtar[1] · Waleed M. Mohamed[2] · Engy El-Shafeiy[1] · Eman M. G. Younis[2]

Received: 18 September 2024 / Accepted: 19 March 2025 / Published online: 9 May 2025

Abstract

Nowadays, social media, especially Twitter, represents a goldmine for understanding consumer opinions about phone brands and applications. Decision-makers can use these insights to address negative views on products and brands and improve the quality of their brands or products to remain competitive. However, while Twitter is a valuable source for representing consumer opinions about phone brands and products, collecting all consumer opinions about specific products or brands and evaluating them as positive, negative, or neutral is challenging. In this paper, a new machine learning-based sentiment analysis system is developed and designed to identify consumer opinions about phone brands, products, and applications, classifying them as neutral, positive, or negative using the Twitter platform. To achieve this, the system consists of four main steps: preprocessing, word categorization, classification, and hyperparameter tuning. In the preprocessing step, each tweet is analyzed to remove hyperlinks, mentions, hashtags, and stopwords that do not affect the nature of the tweet. A new green hybrid categorization mechanism is then developed to perform stemming and lemmatization on each word in the tweets, estimate the importance of each word, and calculate the frequency of each word in the dataset. To do this, the bag-of-words algorithm is used to collect the words present in the dataset, calculate the frequency of each word, and highlight the importance of each word for analyzing the tweet's sentiment. The term frequency-inverse document frequency (TF-IDF) algorithm is then applied to the bag-of-words results to estimate the relevance of each word to each tweet. Finally, the singular value decomposition algorithm is applied to the TF-IDF results to perform word factorization of the entire dataset by computing the singular matrix and left singular vector for each tweet, which provides the significance of each word and identifies the direction of the words in each tweet. Each tweet is then classified as neutral, positive, or negative using a new ensemble algorithm that combines neural networks, K-nearest neighbors (KNN), and random forest in the classification stage. To enhance system performance, the optimal hyperparameters for the neural network, KNN, and random forest algorithms are identified using a modified version of the walrus metaheuristics algorithm. The performance of the proposed system is tested on a dataset named "Brands and Product Emotions," which contains 9094 tweets reflecting consumer opinions about phone brands and products collected from the CrowdFlower platform. The system outperforms state-of-the-art techniques by achieving an accuracy of 99.84%. Future applications of this system include real-time sentiment monitoring for brand reputation management, enhanced customer feedback analysis for product development, and integration with **c**ustomer **r**elationship **m**anagement systems to tailor marketing strategies. The promising results demonstrate the potential for deploying this system in various consumer and business domains.

https://doi.org/10.1007/s00521-025-11194-6

Keywords Sentiment analysis · Ensemble algorithm · Neural network · Term frequency-inverse document frequency · Phone brands · Bag-of-words

Abbreviations

ANN	Artificial neural network
ANN-GA	Artificial neural network with genetic algorithm
BERT	Bidirectional encoder representations from transformers
CNN	Convolutional neural network
IDF	Inverse document frequency
KNN	K-nearest neighbors
MAE	Mean absolute error
ML	Machine learning
DL	Deep learning
AI	Artificial intelligence
IGWO	Improved grey wolf optimizer
MLP	Multilayer perceptron
MSE	Mean squared error
NB	Naïve Bayes
POS	Part of speech
RF	Random forest
SVD	Singular value decomposition
SVM	Support vector machine
TF	Term frequency
TF-IDF	Term frequency-inverse document frequency
URL	Uniform resource locator
EGJO	Enhanced golden jackal optimizer

1 Introduction

Every day, individuals express their opinions about iPhone and Android brands and applications on social media [1]. These opinions represent a valuable resource for decision-makers to understand the positive and negative perceptions of their products or brands [2]. As a result, social media sentiment analysis plays a crucial role in the highly competitive smartphone industry [3]. Analyzing consumer sentiment allows decision-makers to identify the strengths and weaknesses of their brands and products by examining public opinion, which can influence the future development of these brands with the goal of improving product quality and enhancing customer satisfaction and loyalty [4]. The field of sentiment analysis, a subfield of natural language processing (NLP), can be used to analyze consumer sentiment [5]. Sentiment analysis is a powerful tool for classifying sentiments into positive, negative, or neutral opinions [6]. In recent years, sentiment analysis has been utilized to understand consumer behavior through social media [6]. Studies show that applying sentiment analysis techniques increases the ability to monitor brands and products and obtain feedback, which can lead to a 15% increase in customer satisfaction scores [7, 8].

To extract consumer opinions about smartphone products and brands, various social media platforms like Facebook, Twitter, and Instagram are utilized. However, Twitter is considered the essential platform for sentiment analysis because it provides decision-makers with vast, real-time data on the opinions of millions of consumers. Twitter generates approximately 500 million tweets daily, offering unique and real-time insights into public sentiment on a wide range of topics [9]. This massive volume of tweets makes Twitter a rich source for understanding the importance and strengths of brands and products, as well as the issues they face. Despite Twitter's value as a tool for sentiment analysis, it also presents challenges [10]. Each tweet often contains content such as slang, hashtags, links, emojis, and abbreviations, which are not always relevant to sentiment analysis [10]. Additionally, Twitter's concise nature, with its 280-character limit per tweet, poses a significant challenge in understanding sentiment from very short sentences [11]. Overall, Twitter is considered an ideal platform for identifying the sentiments and emotions of individuals through brief and informal tweets [12].

The process of sentiment analysis on Twitter data presents a formidable challenge due to the difficulty of managing, analyzing, and processing the vast number of tweets generated daily [13]. In 2020, a study showed that Twitter represents one of the fastest-growing and largest sources of unstructured data available [14]. The massive and rapidly generated real-time data from Twitter demands robust techniques to handle the influx without losing any insights. This includes capturing the specific words and details in each tweet that indicate particular opinions [15]. Ayvaz and Shiha reported that 20% of Twitter data is written in a way that obscures and hides the intended sentiment, making analysis a significant challenge [16].

Due to Twitter data's anonymous and unstructured nature, it is difficult to collect and analyze all the data using traditional methods. Traditional approaches, such as lexicon-based methods, rely on predefined lexicons and manual analysis to understand each word in tweets. This method often lacks accuracy in identifying sentiment, particularly for informal text that includes slang or words not found in dictionaries [17]. Additionally, Nazman et al. [17] showed that 40% of tweets published on Twitter are written in non-standard language, complicating traditional analysis methods.

Some tweets also contain ambiguous sentiments or mixed emotions, making it difficult to judge sentiment using traditional and automated analysis methods [18]. These tweets must be evaluated within the context of the entire tweet to accurately determine sentiment [19, 20]. A study in 2019 found that 16% of tweets contain ambiguous sentiments and sarcasm [21]. Recent work has focused on developing and designing automated tools based on machine learning and deep learning techniques to identify consumer sentiment about brands and products from Twitter data. Although these recent approaches achieve acceptable performance, they suffer from several drawbacks: (1) the ambiguous nature of tweet sentiments makes it challenging for recent methods relying on deep learning algorithms, such as support vector machines (SVM), Naive Bayes, and k-nearest neighbors (KNN), to achieve high accuracy, (2) while recent methods that use deep learning techniques for classifying each tweet achieve acceptable accuracy, they often consume considerable time to process each tweet, and (3) methods that depend on categorizing each word in tweets into appropriate tokens or lemmas take a long time to perform tokenization or categorization.

Therefore, it is crucial to address the drawbacks of previous work by proposing a new sentiment analysis system that can classify tweets about phone brands and products into positive, negative, or neutral opinions using machine learning algorithms. This system aims to categorize each word in the tweets more efficiently and classify each tweet more quickly. The proposed system will handle slang, abbreviations, and ambiguous sentiments, potentially achieving higher accuracy. This paper presents several contributions to sentiment analysis, such as:

1. Designed and implemented a new sentiment analysis tool for phone products and brands on the Twitter platform that classifies each tweet as a positive, negative, or neutral opinion about the brand or product.
2. Proposed a new hybrid categorization tool that categorizes each word in the tweets and calculates the frequency of each word in the dataset based on a combination of Bag-of-words, Term frequency-inverse document frequency (TF-IDF), and singular value decomposition (SVD) algorithms.
3. Proposed a new ensemble classification model that combines k-nearest neighbors (KNN), random forest, and neural networks.
4. Developed a modified version of the Walrus optimizer to identify the optimal hyperparameter values for the KNN, random forest, and neural network models.

Table 1 Related work comparison

Related work	Algorithm(s) Used	Optimization algorithm	Dataset	Results
Anjum et al. [22]	SVM, Naïve Bayes, Maximum Entropy, and Logistic Regression	–	The author used five datasets: Sentiment Analysis 100,000 tweets [23], 1000 Tweets [24], Amazon Labeled Dataset [25], Yelp Dataset [26], and Amazon Mobiles Dataset [27], collected from different sources	Logistic Regression outperformed the other algorithms, achieving accuracy rates of 93.74%, 91.2%, 77.2%, 80%, and 77.82% when applied to the Sentiment Analysis 100,000 tweets, 1000 Tweets, Amazon Labeled Dataset, Yelp Dataset, and Amazon Mobiles Dataset, respectively
Shaheen et al. [28]	Random forest, CNN, SVM, Multinomial Naïve Bayes, Gradient Boosting, and LSTM	–	The authors used a dataset collected from Kaggle, containing 400,000 individual consumer reviews of unlocked phones on the Amazon platform	Their experimental results showed that the random forest classifier outperformed the other classifiers, achieving 85% accuracy
Kayakuş et al. [29]	SVM	–	The authors tested their system on a dataset containing consumer reviews of the iPhone 11, collected from Trendyol, a very popular platform in Turkey	Their experimental results show that their system achieved a recall of 0.963 and an accuracy of 0.500
Hossain and Rahman [30]	Decision Tree, KNN, SVM, Logistic Regression, and random forest	–	The authors collected a dataset from the Yelp website containing individual reviews and ratings about products	The authors' experimental results show that Logistic Regression outperformed other machine learning classifiers, achieving 93.77% accuracy
Ma'ruf et al. [31]	Ensemble algorithm, which is a combination of KNN, SVM, and Naïve Bayes		The authors collected a dataset consisting of 510 consumer reviews about smartphones from Shopee and Tokopedia platforms	Their experimental results showed that the ensemble algorithm outperformed the individual algorithms (KNN, SVM, and Naïve Bayes), achieving 89.22% accuracy when the data was split into 80% training and 20% testing. Additionally, the results indicated that KNN outperformed the other machine learning algorithms and the ensemble algorithm, achieving 91.18% accuracy when the data was divided into 70% for training and 30% for testing
Lagrari and Elkettani [32]	A modified version of the BERT model	LA metaheuristic algorithm	The authors use the Brands and Product Emotions dataset	Their system outperformed CNN, Naïve Baise, and BERT algorithms, achieving an accuracy of 92.2%
Iqbal et al. [34]	LSTM	–	The authors collected five datasets: Yelp, Amazon Fine Food Reviews, Cell Phones and Accessories, Amazon Products, and IMDB from different sources	The model that used an embedded layer and dense layer with a sigmoid activation function outperformed the other two models, achieving accuracy rates of 87%, 87%, 97%, 75%, and 83% on the Amazon Fine Food Reviews, Cell Phones and Accessories, Amazon Products, IMDB, and Yelp datasets, respectively
Kaur and Sharma [35]	LSTM	–	The authors used three different datasets: SemEval-2014, Sentiment140, and STS-Gold, which were collected from various sources and contain information about brands and products	The authors' experimental results demonstrated the efficiency of their system, achieving 95.9% accuracy when applied to the SemEval-2014 dataset
Alzahrani et al. [36]	LSTM and CNN-LSTM, a hybrid model combining CNN and LSTM algorithms	–	The authors collected a dataset containing 9,500 individual reviews about phone, tablet, television, and laptop brands and products from the Amazon website	Their experimental results showed that LSTM outperformed the CNN-LSTM algorithm, achieving 94% accuracy
Rasappan et al. [37]	LSTM	EGJO and IGWO	The authors used a dataset collected from Prompt Cloud, containing 400,000 reviews with 3 and 4-star ratings about unlocked mobile phones from Amazon	The authors' experimental results show that their system achieved high accuracy, with a result of 97.65%

The rest of the paper is organized as follows: Sect. 2 discusses some of the recent works in the field. The proposed system is detailed in Sect. 3. In Sect. 4, the dataset used is presented, and the proposed system is evaluated and compared with recent work, with the results discussed. Finally, the paper is concluded, and future work is presented in Sect. 5.

2 Related work

Many recent works have been developed for sentiment analysis systems focusing on brands and products. Some of these works use machine learning algorithms to identify individuals' sentiments toward brands and products. For example, Anjum et al. [22] proposed a brand sentiment analysis system that classifies opinions as either negative or positive using machine learning algorithms. They removed stop words from each opinion and used an N-gram algorithm to identify relationships between words within a sentence. The authors employed four machine learning algorithms—support vector machine (SVM), Naïve Bayes, maximum entropy, and logistic regression—to classify each opinion as positive or negative. They tested their system on five datasets: Sentiment Analysis 100,000 tweets [23], 1000 Tweets [24], Amazon Labeled Dataset [25], Yelp Dataset [26], and Amazon Mobiles Dataset [27], collected from different sources to evaluate the performance of their system. The experimental results showed that logistic regression outperformed the other algorithms, achieving accuracy rates of 93.74%, 91.2%, 77.2%, 80%, and 77.82% when applied to the Sentiment Analysis 100,000 tweets, 1000 Tweets, Amazon Labeled Dataset, Yelp Dataset, and Amazon Mobiles Dataset, respectively.

Shaheen et al. [28] proposed a sentiment analysis system for reviews with 3 or 4-star ratings that individuals have left on unlocked phones on the Amazon platform. This system classifies each review as either positive or negative. The authors divided their system into three main steps: preprocessing, analysis, and classification. In the preprocessing step, the authors removed stop words and punctuation, converted each word in the review to its stem form, and used the part of speech (POS) algorithm to convert each stem to its morphological form. In the analysis phase, the review was checked to determine if it was an undesirable review. Finally, the authors used a random forest algorithm to classify each review as positive or negative. To test their system, they used a dataset collected from Kaggle, containing 400,000 individual consumer reviews of unlocked phones on the Amazon platform. They also tested the performance of their system against several machine and deep learning algorithms, including convolutional neural networks (CNN), support vector machines (SVM), Multinomial Naïve Bayes, gradient boosting, and long short-term memory (LSTM). The experimental results showed that the random forest classifier outperformed the other classifiers, achieving 85% accuracy.

Kayakuş et al. [29] proposed a machine learning sentiment analysis system capable of identifying individuals' opinions about the iPhone 11 as positive, negative, or neutral. Their system follows four steps: preprocessing, text mining, feature extraction, and classification. In the preprocessing step, they tokenize each word in the reviews by breaking down each review into meaningful words and finding the lemma for each word. They also removed unnecessary symbols, punctuation, and hashtags. In the text mining step, each word in the review is analyzed to uncover hidden meanings. The authors then applied the N-grams algorithm to extract features from the review by finding the relationship between each word and its neighboring words. Finally, the SVM algorithm is used to classify each review as negative, positive, or neutral. To evaluate the performance of their system, the authors tested it on a dataset containing consumer reviews of the iPhone 11, collected from Trendyol, a very popular platform in Turkey. The experimental results show that their system achieved a recall of 0.963 and an accuracy of 0.500.

Ma'ruf et al. [31] designed and developed a smartphone sentiment analysis system that can identify consumer sentiment about smartphones through their reviews. The authors collected a dataset consisting of 510 consumer reviews about smartphones from Shopee and Tokopedia platforms. The text preprocessing involved converting all capital letters to lowercase, and removing punctuation and irrelevant words that might affect the sentiment analysis process. The TF-IDF method was then used to convert words into numerical vector forms. Finally, the authors proposed a new ensemble algorithm that classifies each text using SVM, Naïve Bayes, and KNN, followed by majority voting to determine the final decision for each text. The experimental results showed that the ensemble algorithm outperformed the individual algorithms (KNN, SVM, and Naïve Bayes), achieving 89.22% accuracy when the data was split into 80% training and 20% testing. Additionally, the results indicated that SVM outperformed the other machine learning algorithms and the ensemble algorithm, achieving 91.18% accuracy when the data was divided into 70% for training and 30% for testing.

Due to the low accuracy achieved by previous works, some recent efforts have shifted toward using deep learning algorithms to achieve higher accuracy. For example, Lagrari and Elkettani [32] proposed a sentiment analysis system based on deep learning techniques to identify opinions about phone brands and products using Twitter data. Their system classifies each tweet as neutral, negative, or positive regarding phone brands and applications. The authors designed their system to go through three main steps: preprocessing, tokenization, and classification. In the preprocessing step, they used a bag-of-words algorithm to convert each tweet's words to their stems or roots and removed stop words and white spaces. They then used an n-grams algorithm to identify the relationships between words and their neighbors, classifying each word as unigrams, bigrams, trigrams, or n-grams. Finally, the authors developed a new version of the bidirectional encoder representations from transformers (BERT) model, using the Standard Lion Algorithm (LA) to update the maximum sequence length of the BERT encoder. The LA algorithm was also used to identify the optimal hyperparameters of the BERT model. To test their system's performance, the authors used the Brands and Product Emotions dataset, which contains 9094 tweets about phone brands and products collected from Twitter, available through the CrowdFlower website [33]. They also trained and tested their system using other machine and deep learning algorithms, such as CNN, BERT model, Naïve Bayes, and sequential minimal optimization. Their system outperformed other deep learning and machine learning algorithms, achieving an accuracy of 92.2%.

Iqbal et al. [34] proposed a sentiment analysis system that classifies consumer reviews, tweets, posts, and comments on brands and phones into positive or negative opinions about the products and brands using the LSTM deep learning algorithm. Their system involves three main steps: preprocessing, feature encoding, and classification. In the preprocessing step, the authors removed punctuation, hashtags, and stop words, and used the POS algorithm to tokenize each word according to its grammatical form. Next, they used a word embedding algorithm to encode words with similar meanings together and convert each text into a fixed-size vector during the feature encoding step. Finally, the authors proposed three different LSTM architectures with various layers and configurations to classify each text as either a positive or negative opinion. To test their system's performance, the authors collected five datasets: Yelp, Amazon Fine Food Reviews, Cell Phones and Accessories, Amazon Products, and IMDB from different sources. When applied to these datasets, the model that used an embedded layer and dense layer with a sigmoid activation function outperformed the other two models, achieving accuracy rates of 87%, 87%, 97%, 75%, and 83% on the Amazon Fine Food Reviews, Cell Phones and Accessories, Amazon Products, IMDB, and Yelp datasets, respectively.

Kaur and Sharma [35] designed and developed a brand and product sentiment analysis system capable of identifying an individual's sentiment as positive, negative, or neutral using natural language processing techniques and an LSTM model. Their system operates through three main steps: preprocessing, feature extraction, and classification. In the preprocessing step, the authors employed natural language processing techniques to remove unnecessary elements from the reviews, such as stop words, words with fewer than three characters, and hashtags. In the feature extraction step, a new hybrid method was used to extract features from the preprocessed text. This method combines review-related features, which identify the polarity of each word (such as emoticons and negations), with aspect-related features, which address the issues of ambiguous and sarcastic words. The results from these two methods are then combined and fed into the LSTM model to make the final decision for each review. The performance of their system was evaluated using three different datasets: SemEval-2014, Sentiment140, and STS-Gold, which were collected from various sources and contain information about brands and products. The authors' experimental results demonstrated the efficiency of their system, achieving 95.9% accuracy when applied to the SemEval-2014 dataset.

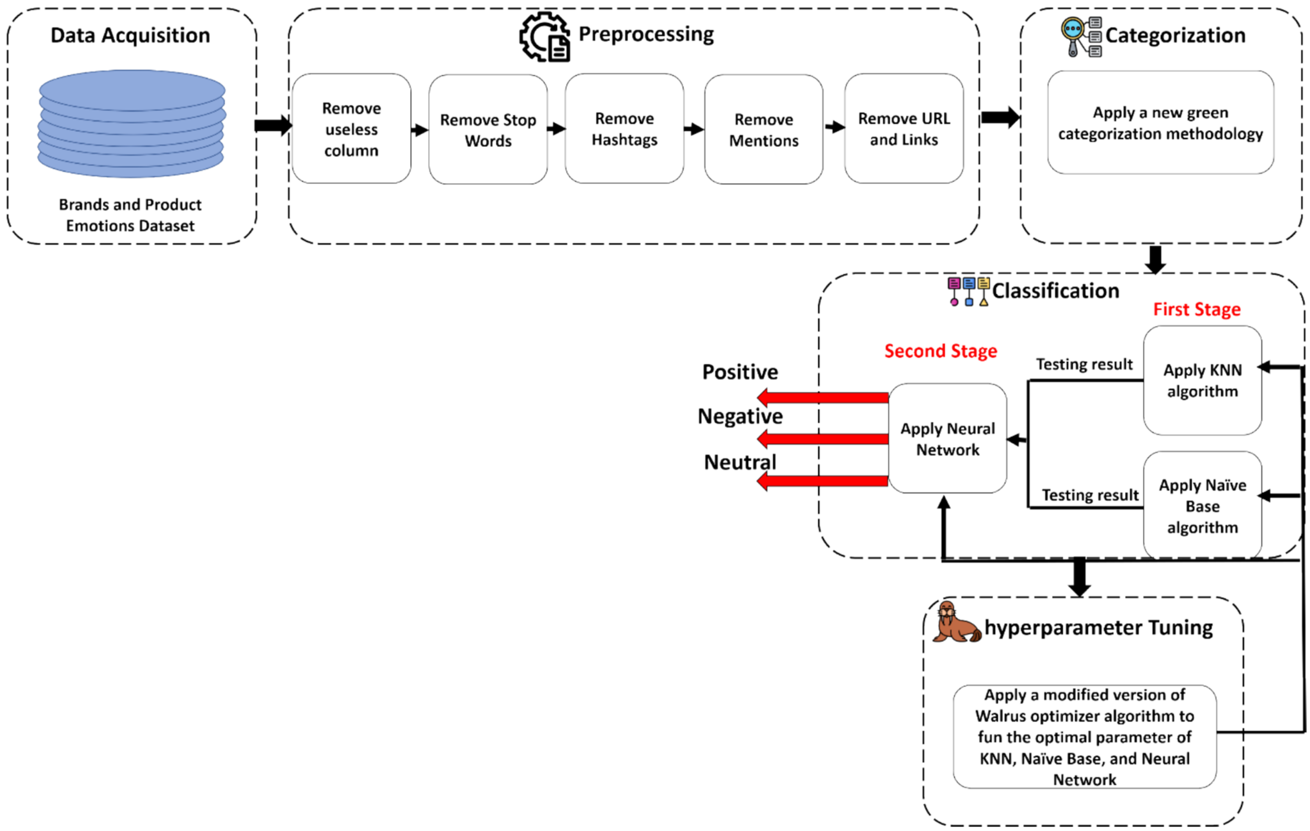

Fig. 1 proposed system framework

Alzahrani et al. [36] proposed a deep learning sentiment analysis system capable of classifying consumer reviews about e-commerce products and brands into positive or negative categories. They collected a dataset containing 9500 individual reviews about phone, tablet, television, and laptop brands and products from the Amazon website. Their system classifies each review by first removing stop words and punctuation, then converting each word to its root form. A POS algorithm is applied to identify the grammatical form of each word. A score value is calculated for each review based on a lexicon, with the review marked as positive if the score is greater than 0 and negative otherwise. Afterward, the Keras text tokenizer algorithm is used to compute word embeddings for each word and convert the reviews into numerical representations. Finally, LSTM and CNN-LSTM, a hybrid model combining CNN and LSTM algorithms, are used to classify each review as positive or negative. The experimental results showed that LSTM outperformed the CNN-LSTM algorithm, achieving 94% accuracy.

Rasappan et al. [37] designed and implemented a new sentiment analysis system called the enhanced golden jackal optimizer-based long short-term memory (EGJO-LSTM), capable of classifying consumers' reviews about e-commerce products, specifically phone brands and products, as positive, negative, or neutral. Their system follows three main steps: preprocessing, feature selection, and classification. In the preprocessing step, the authors remove stop words, punctuation, rows with null data, and special symbols and convert each word into its root form. Next, in the feature selection step, they propose the Log-term Frequency-based Modified Inverse Class

Table 2 Shows the number of tweets for each category

Sentiment category	Number of tweets in the dataset
Neutral	5544
Positive	2978
Negative	570

Table 3 Shows some examples of the dataset before and after the preprocessing stage

Tweets before preprocessing	*Tweets after preprocessing*	*Sentiment category*
Very smart from @madebymany #hollergram iPad app for #sxsw! http://t.co/A3xvWc6 *(I may leave my Vuvuzela at home now)*	Very smart from the iPad app! (I may leave my vuvuzela home now)	Positive emotion
@jessedee Know about @fludapp? Awesome iPad/iPhone app that you'll likely appreciate for its design. Also, they're giving free Ts at #SXSW	Know about? Awesome iPad/iPhone app you'll likely appreciate its design. Also, they're giving free Ts	Positive emotion
@sxsw I hope this year's festival isn't as crashy as this year's iPhone app. #sxsw	I hope year's festival isn't crashy year's iPhone app	Negative emotion
#sxsw day 1-Marissa Mayer: Google Will Connect the Digital & Physical Worlds Through Mobile {link}	Boooo! RT Flipboard developing iPhone version, Android, says	Negative emotion
Any other #Sxsw accounts I need to follow or apps to download for iPhone?	Any other accounts I need follow apps download iPhone?	Neutral
@teachntech00 New iPad Apps For #SpeechTherapy And Communication Are Showcased At The #SXSW Conference http://ht.ly/49n4M *#iear #edchat #asd*	New iPad Apps Communication Showcased Conference	Neutral

Frequency (LF-MICF) algorithm to assign a weight to each word. The improved grey wolf optimizer (IGWO) algorithm is also used in this step to select the best features that can discriminate between the three labels: positive, negative, and neutral. Finally, the proposed EGJO-LSTM model, a combination of the enhanced golden jackal optimizer (EGJO) and LSTM, is used to classify each review. To evaluate their system, they used a dataset collected from Prompt Cloud, containing 400,000 reviews with 3- and 4-star ratings about unlocked mobile phones from Amazon. The authors' experimental results show that their system achieved high accuracy, with a result of 97.65%.

Table 1 summarizes the related work and compares them by showing the machine learning and deep learning algorithms, datasets, and results used in each study. Table 1 also indicates whether a metaheuristic algorithm was employed.

As shown in Table 1, the recent works demonstrate their efficiency but suffer from some disadvantages, such as (1) related works that rely on machine learning algorithms achieve low accuracy, (2) related works that rely on deep learning algorithms take a long time to identify the sentiment of each text, and (3) a few related works that design a sentiment analysis system capable of identifying user opinions on all phone brands and applications.

3 Methodology

In this paper, a new green sentiment analysis system is proposed to identify individuals' sentiments about mobile and phone brands on the Twitter platform as either neutral, negative, or positive opinions. The proposed system goes through five main stages: data acquisition, preprocessing, categorization, classification, and hyperparameter tuning to accurately classify individuals' sentiments. In the data acquisition stage, a mobile brands and product dataset is collected to evaluate the performance of the proposed system. During the preprocessing steps, all useless information in the tweets, such as stop words, hashtags, and mentions, which do not serve as indicators for classifying sentiments, is removed. The remaining parts of the tweets are then categorized and converted to their roots to assess the importance of each word in the categorization phase. In the classification phase, a new ensemble algorithm is proposed and applied to classify each tweet as positive, neutral, or negative with respect to each phone brand and product. Finally, a modified version of the walrus optimizer algorithm is applied to identify the optimal hyperparameters that enhance the performance of our ensemble algorithm. Figure 1 shows the framework and the stages that the proposed system passes through.

3.1 Data acquisition

In this paper, the proposed system's performance is evaluated using the dataset named Brands and Product Emotions collected from the data world platform, which is a free repository of numerous datasets [33]. This dataset consists of 9093 different individual opinions about mobile brands and products (Android and iPhone) and contains three attributes: tweettext, emotion_in_tweet_is_directed_at, and is_there_an_emotion_directed_at_a_brand_or_product for each individual opinion collected from the Twitter platform. The tweettext attribute contains the tweet that each individual wrote on the Twitter platform to express their opinion about the products. The is_there_an_emotion_directed_at_a_brand_or_product attribute represents the category of sentiment for each tweet, while the emotion_in_tweet_is_directed_at attribute represents the mobile brand or product category. Each tweet in the dataset is classified as neutral, positive, or negative. Table 2 shows the frequency of each category in the dataset. The number of tweets classified as neutral, positive, and negative is 5544, 2978, and 570, respectively, as shown in Table 2. Table 3 shows some examples of tweets from the dataset.

3.2 Preprocessing

As shown in Table 3, unfortunately, the individual sentiments in the dataset are expressed in tweets that contain irrelevant information that does not affect sentiment identification, such as hashtags, URLs, and mentions. Therefore, this dataset must be preprocessed to remove this unnecessary data. Algorithm 1 outlines the steps we performed to preprocess and clean the dataset. As shown in Fig. 1 and Algorithm 1, during the preprocessing stage, columns that do not impact the sentiment classification process are removed. For example, the dataset contains the emotion_in_tweet_is_directed_at attribute, which represents the category of the phone brand or product mentioned in the tweet. Since this attribute does not influence the classification stage, it is removed.

Additionally, when analyzing the dataset, we found many tweets containing mentions, hashtags, and URLs. To address this, we created regular expressions to remove them: '#\w' for hashtags, '@\w' for mentions, and 'http\S' and 'www.\S' for URLs and links, respectively. We also discovered tweets that included the placeholder {link}, indicating a link, and used the regular expression '{link}' to remove these as well.

Moreover, we identified many stop words such as "a," "an," "and," "are," "as," "at," "be," "but," "by," "for," "if," "in," "into," "is," "it," "no," "not," "of," "on," "or," "such," "that," "the," and "their." These stop words do not effect on sentiment classification and would unnecessarily increase processing time in the categorization and classification stages. To minimize this processing time, we removed stop words from each tweet.

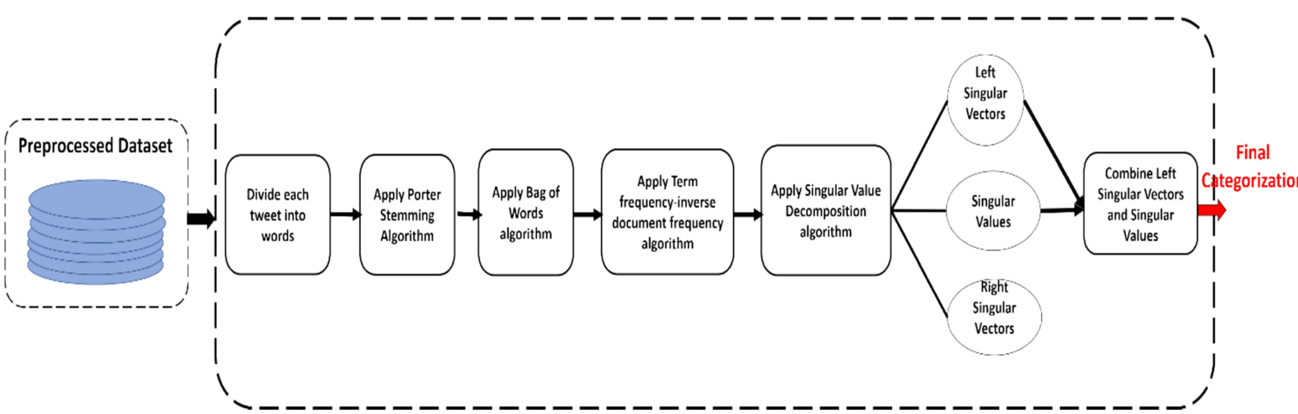

Fig. 2 proposed green categorization method

Table 3 also shows the results of some examples after applying Algorithm 1 to the dataset, demonstrating the removal of hashtags, mentions, URLs, links, and stop words. The table also illustrates that the length of each tweet is reduced, which helps save time in the subsequent stages.

Algorithm 1 Preprocessing algorithm

```
Input: Dataset (DA)
Output: Cleaned dataset (CD)
    Removing useless attributes
    For I=1 : 9093
        Remove the hashtags from each I tweet using the '#\w' regular expression
        Remove the mention from each I tweet using the '@\w' regular expression
        Remove links from each I tweet using the 'http\S' regular expression
        Remove the URL from each I tweet using the 'www.\S' regular expression
        Remove the word {link} from each I tweet using '{link}' regular expression
        Remove the stop words from each I tweet
    End For
    Store the result dataset in a Cleaned dataset (CD)
Return CD
```

3.3 Categorization

After the dataset is cleaned, each word must be categorized to show its importance in the classification stage. A new hybrid method is used to assign a weight to each word in every tweet, making the classification stage easier. This method relies on traditional approaches that are time-efficient, unlike methods that depend on deep learning algorithms. Figure 2 illustrates the new proposed categorization method and the steps the system follows to categorize each word in the dataset. As shown in Fig. 2, the new categorization method consists of four main subphase: word stemming, word frequency, word weight, and statistical analysis. Before applying the four subphase, each tweet is divided into individual words.

3.3.1 Word stemming

In the word stemming subphase, the Porter Stemming Algorithm is applied to find the root or stem of each word in every tweet. The Porter Stemming Algorithm is considered one of the most efficient algorithms for converting words to their root forms because it applies a set of grammar rules for root form transformation. It converts all variations of a word to a single representation, which subsequently enhances the accuracy of the system in sentiment classification. For example, it converts all forms of the word "happy," such as "happier," "happiest," and "happily," to the root form "happi." Beyond its efficiency, the Porter Stemming Algorithm can quickly process large datasets, reducing noise by simplifying the vocabulary and minimizing the number of unique words. This simplification makes sentiment classification easier and more accurate. The word stemming algorithm that used in this paper is presented in Algorithm 2.

Algorithm 2 Word Stemming Algorithm

```
Input: Dataset (DA), Tweets Number (N)
Output: Root Form (RF)
    For I=1 : N
        Divide I tweet to list of words.
        For J=1 : size (I tweet)
            If J word end with "sses"
                RF (I,J) = Remove "es" from the end of the word
            Else If J word end with "ies"
                RF (I,J) = Remove "ies" from the end of the word
            Else if J word end with 's'
                RF (I,J) = Remove 's' from the end of the word
            End If
            If J word end with "ed" or "ing"
                If J word contain Vowel (aeiou)
                    RF (I,J) = Remove "ing" or 'ed' from the end of the word
                End If
            End If
            If J word end with "eed"
                C=Count number of constant character sequence in J word
                If C > 0
                    RF (I,J) = Remove "eed" from the end of the word
                End If
            Else If J word end with "ed"
                RF (I,J) = Remove "eed" from the end of the word
                If RF (I,J) contain Vowel (aeiou)
                    Swich (end of RF (I,J))
                        Case "at":  RF (I,J)= RF (I,J) + 'e'
                        Case "bl": RF (I,J)= RF (I,J) + 'e'
                        Case "iz": RF (I,J)= RF (I,J) + 'e'
                    End Switch
                End IF
            Else If J word end with " ing "
                RF (I,J) = Remove "ing" from the end of the word
                If RF (I,J) contain Vowel (aeiou)
                    Switch (end of RF (I,J))
                        Case "at":  RF (I,J)= RF (I,J) + 'e'
                        Case "bl": RF (I,J)= RF (I,J) + 'e'
                        Case "iz": RF (I,J)= RF (I,J) + 'e'
                    End Switch
                End IF
            End if
            If J word end with "ational"
                RF (I,J) = remove "ational"
                RF (I,J) = RF (I,J) + "ate"
            Else if J word end with " tional"
                RF (I,J) = remove " tional "
                RF (I,J) = RF (I,J) + "tion"
            Else if J word end with "enci "
```

```
        RF (I,J) = remove " enci"
        RF (I,J) = RF (I,J) + " ence "
Else if J word end with " anci"
        RF (I,J) = remove " anci"
        RF (I,J) = RF (I,J) + " ance"
Else if J word end with " izer"
        RF (I,J) = remove " izer"
        RF (I,J) = RF (I,J) + " ize "
Else if J word end with " abli "
        RF (I,J) = remove " abli"
        RF (I,J) = RF (I,J) + "able"
Else if J word end with " alli"
        RF (I,J) = remove " alli"
        RF (I,J) = RF (I,J) + "al"
Else if J word end with "entli"
        RF (I,J) = remove "entli"
        RF (I,J) = RF (I,J) + "ent"
Else if J word end with " eli"
        RF (I,J) = remove " eli"
        RF (I,J) = RF (I,J) + "e"
Else if J word end with " ousli"
        RF (I,J) = remove "ousli"
        RF (I,J) = RF (I,J) + " ous"
Else if J word end with " ization"
        RF (I,J) = remove " ization"
        RF (I,J) = RF (I,J) + " ize"
Else if J word end with " ation"
        RF (I,J) = remove " ation"
        RF (I,J) = RF (I,J) + " ate"
Else if J word end with " ator"
        RF (I,J) = remove " ator"
        RF (I,J) = RF (I,J) + " ate"
Else if J word end with " alism"
        RF (I,J) = remove " alism"
        RF (I,J) = RF (I,J) + " al"
Else if J word end with " iveness"
        RF (I,J) = remove " iveness"
        RF (I,J) = RF (I,J) + " ive"
Else if J word end with " fulness"
        RF (I,J) = remove " fulness"
        RF (I,J) = RF (I,J) + " ful"
Else if J word end with " ousness"
        RF (I,J) = remove " ousness"
        RF (I,J) = RF (I,J) + " ous"
Else if J word end with " aliti"
        RF (I,J) = remove " aliti"
        RF (I,J) = RF (I,J) + " al"
Else if J word end with " iviti"
        RF (I,J) = remove " iviti"
```

```
                    RF (I,J) = RF (I,J) + " ive"
            Else if J word end with " biliti"
                    RF (I,J) = remove " biliti"
                    RF (I,J) = RF (I,J) + " bility"
            End If
            If J word end with " icate"
                    RF (I,J) = remove " icate "
                    RF (I,J) = RF (I,J) + " ic"
            Else If J word end with " ative"
                    RF (I,J) = remove " ative"
            Else If J word end with " alize"
                    RF (I,J) = remove " alize"
                    RF (I,J) = RF (I,J) + " al"
            Else If J word end with " iciti"
                    RF (I,J) = remove " iciti"
                    RF (I,J) = RF (I,J) + " ic"
            Else If J word end with " ical"
                    RF (I,J) = remove " ical"
                    RF (I,J) = RF (I,J) + " ic"
            Else If J word end with " full"
                    RF (I,J) = remove " ful"
            Else If J word end with " ness"
                    RF (I,J) = remove " ness"
            End If
            If J word end with "e"
                    C=Count number of constant character sequence in J word
                    If C > 1
                            RF (I,J) = Remove "e" from the end of the word
                    End If
            Else If J word end with "ed" or "ing"
                    C=Count number of constant character sequence in J word
                    If C > 1
                            RF (I,J) = Remove "ed" or "ing" from the end of the word
                    End If
            End If
            If J word end with "y"
                    C=Count number of constant character sequence in J word
                    If C > 1
                            RF (I,J) = Replace "y" with "i" from the end of the word
                    End If
            End If
        End For
    End For
Return RF
```

As shown in Algorithm 2, the grammar rules are applied using a simple "if-then" approach, which does not consume much time to execute.

3.3.2 Word frequency

It is preferable for machine learning algorithms to classify numerical data rather than text. Therefore, the bag-of-words (BoW) algorithm is used to convert text into a numerical form, making it easier for machine learning algorithms to classify each tweet. The BoW algorithm relies on recording the unique vocabulary present in all tweets within the dataset and computing the frequency of each vocabulary item across the entire dataset. By applying a stemming algorithm, the number of unique words is reduced since many words are represented in their root form. This reduction speeds up the BoW process and enhances its effectiveness by focusing on the frequency

Table 4 Optimal values for the hyperparameter for the two-stage classifaction model

Machine learning algorithm	Hyperparameter	Optimal configuration
KNN	Num of Neighbors	5
	Distance method	Euclidean Distance
Random forest	Number of trees	150
	Min leaf size	20
Neural network	learning rate	0.0150
	Number of hidden layers	3
	Number of neurons in first layer	39
	Number of neurons in second layer	19
	Number of neurons in third layer	3

of repeated words, disregarding their order within each tweet. While the BoW model does not capture the order or contextual meaning of words, it does not remain effective for estimating word frequencies quickly compared to more complex deep learning techniques that analyze word context. Algorithm 3 illustrates how the bag-of-words is applied to the stemmed dataset, resulting from the stemming process. As shown in Algorithm 3, the algorithm starts by collecting all unique vocabulary into a single vector called Voc. The index of each vocabulary word is recorded in Word_index. Then, the frequency of each word in the entire dataset is calculated in the Freq vector. Finally, we iterate through each word in each tweet to record the frequency of each vocabulary item.

Table 5 System performance when dataset is divided into 80% for training and 20% for testing

Machine learning algorithm	Sentiment category	Metrics			
		Precision	Sensitivity	f1-score	Accuracy
KNN	Positive	59.29%	47.33%	52.64%	68.83%
	Negative	43.75%	20.79%	28.19%	
	Neutral	73.3%	84.70%	78.59%	
Decision tree	Positive	75.20%	76.83%	76.01%	80.70%
	Negative	47.06%	7.92%	13.56%	
	Neutral	84.02%	89.36%	86.61%	
SVM	Positive	100%	100%	100%	94.4475%
	Negative	11. 13%	5.22%	18.41%	
	Neutral	91.71%	94.54%	95.68%	
Random forest	Positive	99.66%	97.33%	98.48%	93.5679%
	Negative	13.18%	6.75%	15.15%	
	Neutral	90.67%	100%	95.11%	
Boost Gradient	Positive	79.05%	13.83%	23.55%	64.9808%
	Negative	8.14%	4.05%	12.27%	
	Neutral	64.12%	98.30%	77.61%	
Proposed Two-stage ensemble model	Positive	100%	99.67%	99.83%	99.84%
	Negative	100%	99.01%	99.50%	
	Neutral	99.73%	100%	99.87%	

Algorithm 3 Word Frequency Algorithm

```
Input: Root Form (RF), Number of tweets (N)
Output: Vector of Bag of word (VB)-
        Voc = {}
        Exist=false;
        For I=1 : N
                For J=1: size (I Tweet)
                        For k=1: size(Voc)
                                If J word exist in Voc
                                        Exist= True
                                        Break
                                End If
                        End For
                        If Exit == false
                                Append word J on Voc
                        Else
                                Exit = false
                        End If
                End For
        End For
        Index = 0
        Word_index ={}
        For I=1 : size (Voc)
                Word_index[I]=index
                Index=index +1
        End For
        Ferq={}
        For K=1 : size (Voc)
                For I=1 : N
                        For J=1: size (I Tweet)
                                If (word = tweet J word)
                                        Freq(K)= Freq(K)+1
                                End if
                        End For
                End For
        End For
        VB={}
        For I=1 : N
                For J=1: size (I Tweet)
                        Vec= new vector with size equal to size(Voc)
                         Initialize all elements in Vec to 0
                        For J=1: size (I Tweet)
                                Ind= Word_index[J]
                                Vec[Ind]= Ferq[Ind]
                        End For
                End For
                Append Vec to VB
        End For
Return VB
```

https://doi.org/10.1007/s00521-025-11194-6

3.3.3 Word weight

After computing the frequency of each word in the dataset, the TF-IDF algorithm is used to enhance the representation of each word by calculating its importance within the dataset. The TF-IDF algorithm is considered an effective tool to use after computing word frequency, as it differentiates between words related to phone brands and products and those that are irrelevant. TF-IDF computes both the local and global occurrence of each word in each tweet, making it crucial for identifying meaningful words that can serve as indicators for each sentiment category. To apply the TF-IDF algorithm, three metrics are calculated: term frequency (TF), inverse document frequency (IDF), and TF-IDF.

The TF metric measures the importance of each word within a tweet by dividing the number of times the word occurs by the total number of words in the tweet that includes the word. Equation 1 shows how to measure the TF metric.

$$\mathrm{TF}_{w,t} = \frac{O_{w.t}}{N_t} \tag{1}$$

where $O_{w,t}$ represents the number of occurrences of a specific word in its tweet, and N_t represents the total number of words in that tweet, respectively.

The IDF metric measures the importance and weight of each word across the entire dataset. A word may have a low frequency in individual tweets but could still have a high significance throughout the dataset. The IDF metric is calculated as follows:

$$\mathrm{IDF}_w = \log \frac{N}{(1 + N_w)} \tag{2}$$

where N and N_w represent the total number of tweets in the dataset and the total number of tweets that include the word w, respectively.

Finally, TF-IDF is calculated by multiplying the TF by the IDF for each word in each tweet to reflect the importance and weight of each word both within individual tweets and across the entire dataset. The equation used to compute TF-IDF is:

$$\mathbf{TF} - \mathbf{IDF}_{t,w} = \mathrm{TF}_{w,t} x \mathrm{IDF}_w \tag{3}$$

The algorithm used to compute the weight and importance of each word in the dataset is presented in Algorithm 4. Algorithm 4 demonstrates that three metrics are calculated to determine the weight of each word: TF, IDF, and the resulting TF-IDF score. These metrics work together to provide a comprehensive evaluation of each word’s significance within individual tweets and across the entire dataset.

Algorithm 4 Word weight Algorithm

```
Input: Vector of Bag of word (VB), Number of tweets (N), Vocabulary (Voc)
Output: Matrix of word weight (MWW)
    For I=1 : N
        For J=1: size (I Tweet)
            Compute Term Frequency (TF(I,J)) according to equation 1
        End For
    End For
    For k=1: size(Voc)
        Compute Inverse Document Frequency (IDF(k)) according to equation 2
    End For
    For I=1 : N
        For J=1: size (I Tweet)
            Compute word weight (MWW(I,J)) according to equation 3
        End For
Return MWW
```

3.3.4 Statistical analysis

To improve the performance of the proposed system, the weights of each word resulting from the TF-IDF algorithm are applied to the SVD algorithm to reduce the dimensionality of the vector output from TF-IDF, as many words are considered irrelevant and contribute to noise. The SVD algorithm compresses the information produced by the TF-IDF algorithm to extract the most significant components, which can serve as indicators for each sentiment category during the classification stage, making the classification more accurate in its predictions. As SVD reduces the data's dimensionality, it enables the classification stage to be performed faster and with less resource consumption. Therefore, applying the SVD algorithm after TF-IDF helps uncover the latent semantic structures of each word in the dataset. While TF-IDF shows the importance or weight of each word, SVD identifies the correlations between words and groups them to capture nuances that might be missed when focusing solely on individual word weights. The combination of SVD and TF-IDF ensures that the proposed system can adapt to different types of information present in tweets.

In this step, the results of TF-IDF are analyzed, and words that are considered noise or irrelevant are removed. To perform the SVD algorithm, three components—Left Singular Vectors (L), Singular Values (Σ), and Right Singular Vectors (R)—are computed by decomposing the weight of each word in the tweets. Suppose that the weights of each word in each tweet across the entire dataset are stored in a matrix AAA (with a size of M × N). This matrix is decomposed into three components L, Σ, and R, and can be expressed mathematically by the following equations.

$$A = L\Sigma R$$

The Left Singular Vectors (matrix L) is an M × M matrix that aims to reduce the dimensionality of the matrix resulting from the TF-IDF algorithm by transforming the weight data into a new orthogonal basis while preserving the important features of this data. The columns in the L matrix can be considered as the directions in the original space where the data has the most variance. The left singular vectors can be computed as follows:

$$AA^T L = L\Sigma^2 \tag{4}$$

where A represents the weight matrix for each word in each tweet across the entire dataset and Σ^2 is a diagonal matrix (root of Singular Values) with the squared singular values σ^2.

The singular values (matrix Σ) is an M × N matrix that shows the strength of each singular value in L and R. The singular values matrix is key to reducing dimensionality, as it retains the largest singular values and ignores the smaller ones. The singular values contain the square roots of the nonzero eigenvalues ($\sigma_1, \sigma_2, \sigma_3, \ldots\ldots, \sigma_r$), where r is the minimum value between M and N. The eigenvalues can be computed as follows:

$$\boldsymbol{\sigma_i} = \sqrt{\text{eigen} \quad \text{values}\left(\boldsymbol{AA^T}\right)} \tag{5}$$

The right singular vectors (matrix R) is an N×N matrix where each row represents a right singular vector for matrix A, and each column corresponds to an eigenvector of the matrix AT A. This matrix defines a new orthogonal basis for the row space of A. The rows in the R matrix can be considered as the directions in the original space where the data has the most variance. The Right Singular Vectors can be computed as follows:

$$A^T AR = R\Sigma^2 \tag{6}$$

Algorithm 5 outlines the steps involved in performing the statistical analysis phase. It details the process of calculating the eigenvalues and eigenvectors of A^T A and A A^T. The eigenvalues are then sorted in descending order according to their corresponding eigenvectors to form the left singular vectors and right singular vectors, respectively. Consequently, the singular values are computed by placing the square roots of the eigenvalues of A^T A on the diagonal of the matrix Σ with all off-diagonal elements set to zero.

Algorithm 5 Statistical analysis Algorithm

```
Input: Matrix of word weight (MWW)
Output: Left Singular values (L), singulars values (Σ), Right singulars values (R)
    C =  A^T * A
    D = A * A^T
    Compure eigenvalues (λi)
    r= Min (rows of A, Columns of A )
     For I=1 : r
          λ1(I) = eigen values of C
          R (I) = eigen vector of C
          λ2(I) = eigen values of D
          L (I) = eigen vector of D
          λ1(I)  = Sort λ1(I)  according to corresponding R (I) (descending order)
          λ2(I)  = Sort λ2(I)  according to corresponding L (I) (descending order)
    End For
    For I=1 : R
          S(I) = sqrt( λ1(I))
    End For
    For I=1:M
          For j=1:N
                If I==J
                      Σ(I) = S(I)
                Else
                      Σ(I) = 0
                End If
          End For
    End For
    R(I) = R(I)^T
Return L , Σ , R
```

After computing the left singular vectors, singular values, and right singular vectors, the right singular vectors are ignored. The left singular vectors and singular values are combined to represent the final numerical form for each tweet. All algorithms used in the proposed hybrid green categorization model are designed to be resource-efficient and time-efficient, avoiding the extensive resource and time consumption typically associated with deep learning techniques.

3.4 Classification

As shown in Figure 1, after applying the proposed hybrid green categorization model, each tweet is classified as neutral, positive, or negative during the classification stage. This paper introduces a new hybrid ensemble algorithm based on a combination of machine learning techniques. Our proposed ensemble model integrates KNN, random forest, and neural networks to accurately classify tweets into positive, neutral, or negative sentiments consists of two main stages.

In the first stage, the left singular vectors and singular values, resulting from the hybrid green categorization model, are fed into KNN and random forest algorithms. These algorithms are used to train on 80% of the tweets and test on the remaining tweets individually. The results from KNN and random forest are then combined and fed into a neural network in the second stage, which makes the final classification decision for each tweet, determining if it reflects a negative, positive, or neutral opinion about phone brands and products.

3.5 Hyperparameter tuning

To improve the performance of the proposed two-stage classification, a new modified version of the Walrus Optimizer metaheuristics algorithm is used to identify the optimal hyperparameters for machine learning algorithms in the two-stage classification model. The Walrus Optimizer is a metaheuristic algorithm inspired by the behavior of walruses, which search for resources over a wide area and cooperate with each other to ensure survival. Similar to how walruses explore extensively for resources, the Walrus Optimization Algorithm strikes a balance between exploration and exploitation by thoroughly searching the space for potential solutions (exploration) and then refining the most promising ones (exploitation). This characteristic helps the algorithm avoid getting stuck in local minima, though it may still encounter this problem.

The main advantage of the Walrus Optimizer is its cooperative behavior among agents, allowing them to share positional information and collectively identify potentially optimal areas. Additionally, the algorithm's adaptability allows it to perform well in complex and challenging environments by searching the entire solution space. Thus, the Walrus Optimizer is useful for fine-tuning hyperparameters, as it seeks out the best solution across the entire space and works to improve it into the optimal solution. However, despite these advantages, the Walrus Optimizer has several disadvantages: (1) it is a hypothetical algorithm, lacking empirical evidence and proven effectiveness in real-world problems like sentiment analysis. (2) The algorithm may focus on improving a suboptimal solution, leading to it getting stuck in local optima. (3) Its tendency to search the entire solution space can result in slow convergence, especially if cooperation between agents is not properly managed. (4) When identifying hyperparameters in machine learning algorithms, there is a significant risk of overfitting, as the algorithm may focus too heavily on training data results.

So, in this paper, we propose a new version of the Walrus Optimization Algorithm by applying orthogonal learning techniques to overcome the disadvantages that the original algorithm suffers from. Orthogonal learning is a powerful tool used with optimization algorithms to address issues such as the lack of balance between exploration and exploitation, getting stuck in local optima, and slow convergence rates. In this paper, the orthogonal learning technique is applied to the Walrus Optimizer by using orthogonal vectors to guide the search space, finding the best solution as quickly as possible while ensuring efficient coverage of the search space. This results in a modified version of the Walrus Optimizer with faster convergence.

Additionally, by applying orthogonal learning to the Walrus Optimization Algorithm, the modified version ensures that the search process does not rely on searching in specific directions, avoids repeating the same solutions, and promotes more diverse search patterns, preventing the algorithm from getting stuck in local optima, thus improving the exploration step. Moreover, the orthogonal learning technique helps the Walrus Algorithm reduce the risk of overfitting because it searches a wide area of the search space rather than focusing on a narrow region, making the algorithm less dependent on specific properties of the training set. The modified version of the Walrus Optimization Algorithm is illustrated in Algorithm 6.

Algorithm 6 shows the steps performed to apply orthogonal learning to the Walrus Optimization Algorithm.

Algorithm 6 Modified Walrus optimization Algorithm using orthogonal learning

```
Input: Population Size (N), Maximum Iterations (M), Mutation Rate
Output: Optimal Hyperparameters (OH)
    Fitness function = max (accuracy)
    Best_sol=−∞
    Optimal Hyperparameters (OH)={}
    Generate randomly a population with N walrus agents
     For i=1 : N
        Fit = estimate the accuracy
        If Fit > Best_sol
            Best_sol=Fit
            Best_agent = i
        End if
    End For
    For i=1 to M
        For j=1 to N
            Generate the Orthogonal Vector (OV) according to hyperparameter values
            New_OP=Generate a new hyperparameter for (OV)
            Fit = estimate the accuracy
            If Fit > Best_sol
                Best_sol=Fit
                Best_agent = j
                OH = New_OP
            End if
        End For
        Share OH and agent Best_agent position to all other walrus agent
        For j=1 to N
            Update agent j to best position of Best_agent
        End For
        Create a new agent by Performing mutation between some agents according to the Mutation Rate
        For j=1 to N
            Fit = estimate the accuracy
            Select the agent with high Fit to the next stage
        End For
        If Convergence happened
            Break
        End IF
    End For
Return OH
```

In the first stage, the modified version of the Walrus Optimization Algorithm is used to identify the hyperparameters for the Random Forest algorithm, including the number of trees and the minimum leaf size, as well as the number of neighbors and distance metric in KNN. In the second stage, it is used to determine the hyperparameters for the number of neurons in the three hidden layers and the learning rate for the neural network algorithm.

4 Result and discussion

4.1 Performance metrics

To evaluate the performance of the proposed system, four performance metrics—accuracy, sensitivity, precision, and F1-score—are used. To calculate these metrics, the True Positive (TP_J), True Negative (TN_J), False Positive (FP_J), and False Negative (FN_J) values are determined for each sentiment category J. Therefore, the sensitivity, precision, and F1-score can be calculated for each sentiment category as follows:

$$\text{Sensitivity}_J = \frac{TP_J}{TP_J + FN_J} \tag{7}$$

$$\text{Precision}_J = \frac{TP_J}{TP_J + FP_J} \tag{8}$$

$$\text{F1} - \text{score}_J = \frac{2x\text{TP}_J}{2 \times \ \text{TP}_J + \text{FP}_J + \text{FN}_J} \tag{9}$$

Finally, the accuracy metric can be measured for the entire system using the following equation.

$$\text{Accuracy} = \frac{\sum_{J=1}^{3} \text{TP}_J}{N} \tag{10}$$

where N represents the total number of tweets used in testing.

4.2 Fine-tuning parameter for two-stage classification model

In this paper, a modified version of the Walrus Optimization Algorithm is used to identify the optimal values for the hyperparameters in the two stages of the classification model. The modified version of the Walrus Optimization Algorithm is applied in the first stage of the classification model to determine the number of neighbors and the distance method hyperparameters for the KNN algorithm, as well as the number of trees and the minimum leaf size for the Random Forest algorithm. Table 4 shows that the optimal hyperparameter values for the KNN algorithm are 5 neighbors and the 'Euclidean Distance' as the best distance method. It also indicates that the optimal hyperparameter values for the Random Forest algorithm are 150 for the number of trees and 20 for the minimum leaf size. Additionally, the modified version of the Walrus Optimization Algorithm is applied to the neural network in the second stage of the classification model to determine the hyperparameters for the number of hidden layers, the number of neurons in each hidden layer, and the learning rate. Table 4 also shows that when the Walrus Optimization Algorithm is applied to the neural network, it finds that the number of hidden layers that achieve the best accuracy is 3, with the number of neurons in each hidden layer being 39, 19, and 3 for the first, second, and third hidden layers, respectively. Additionally, Table 4 indicates that the optimal value for the learning rate, as identified by the Walrus Optimization Algorithm for the neural network, is 0.0150.

4.3 Two-stage ensemble classification model performance

The performance of the two-stage classification model and the proposed categorization method was evaluated by testing it on the dataset, dividing the data into 80% for training and 20% for testing. Table 5 presents the results

Table 6 System performance when fourfold and fivefold are used

Fold algorithm	Machine learning algorithm	Sentiment category	Metrics			
			Precision	Sensitivity	f1-score	Accuracy
fourfold	KNN	Positive	75.15% ± 20.9	66.14% ± 24.4	70.2% ± 22.61	80.28% ± 14.39
		Negative	62.75% ± 29.91	49.33% ± 39.61	53.97% ± 36.71	
		Neutral	82.84% ± 11.89	90.48% ± 7.57	86.42 ± % 9.76	
	Decision tree	Positive	72.61%±1.57	76.05 % ± 2.19	74.28% ± 1.57	79.16%±1.94
		Negative	54.72% ± 17.78	9.04% ± 3.6	15.48% ± 5.99	
		Neutral	82.97 % ± 2.05	87.99% ± 1.35	85.40% ± 1.7	
	SVM	Positive	100 ± 0.0	100 ± 0.0	90.27 ± 0.	95.87%±5.06
		Negative	75.02% ± 5.15	73.37% ± 4.01	74.16% ± 9.47	
		Neutral	97.87% ± 3.92	100% ± 0.0	98.89% ± 2.04	
	Boost Gradient	Positive	78.19%± 1.83	14.70% ± 1.15	24.73% ± 1.63	64.71 ±2.88
		Negative	75.82% ± 27.95	0.34% ± 0.41	0.68% ± 0.81	
		Neutral	63.80% ± 3.06	98.17 ± 0.32	77.31% ± 2.34	
	Random forest	Positive	99.93% ± 0.084	96.72% ± 2.14	98.29% ± 1.13	92.68%± 1.02
		Negative	10.45% ± 2.51	7.78% ± 1.23	13.24% ± 3.51	
		Neutral	89.28% ± 1.7	100% ± 0.0	94.33% ± 0.95	
	Proposed two-stage mode	Positive	99.41% ± 0.47	99.29% ± 0.26	99.35% ± 0.28	99.51% ± 0.19
		Negative	99.02% ± 1.62	99.39% ± 0.78	99.2% ± 0.65	
		Neutral	99.61% ± 0.17	99.63% ± 0.20	99.62% ± 0.16	
fivefold	KNN	Positive	75.25% ± 20.33	66.54% ± 24.38	70.45 ± 22.54	80.28%± 14.48
		Negative	62.76 ± 27.79	49.35 ± 39.19	53.69 ± 35.33	
		Neutral	82.89 ± 12.14	90.43 ± 7.36	86.40 ± 9.83	
	Decision tree	Positive	72.66% ± 2.66	76.16% ± 1.14	74.34% ± 1.25	79.16% ± 1.81
		Negative	54.92% ± 13.54	8.68% ± 1.71	14.96% ± 2.97	
		Neutral	82.98% ± 2.27	88.02% ± 1.43	85.42% ± 1.55	
	SVM	Positive	100% ± 0.0	100% ± 0.0	100% ± 0.0	98.56%±2.40
		Negative	80.0% ± 44.72	73.10% ± 43.50	75.83% ± 43.34	
		Neutral	97.85% ± 3.59	100% ± 0.0	98.89% ± 1.87	
	Boost Gradient	Positive	78.14% ± 2.82	14.73% ± 1.66	24.75 % ± 2.35	64.71%±2.63
		Negative	20.0% ± 44.72	0.32% ± 0.70	0.62% ± 1.39	
		Neutral	63.81% ± 2.78	98.17% ± 0.24	77.32% ± 2.06	
	Random forest	Positive	99.93% ± 0.091	96.92% ± 1.74	98.4% ± 0.88	92.73%± 0.96
		Negative	13.83% ± 2.55	12.47% ± 5.77	4.65% ± 6.09	
		Neutral	89.35% ± 1.59	100% ± 0.0	94.37% ± 0.89	
	Proposed two-stage mode	Positive	99.48% ± 0.54	99.80% ± 0.7	99.64% ± 0.27	99.69%± 0.18
		Negative	98.99% ± 1.89	99.57% ± 0.59	99.27% ± 0.86	
		Neutral	99.87% ± 0.05	99.64% ± 0.26	99.76% ± 0.15	

achieved by the proposed method. The table shows that after the classification stage, the two-stage ensemble model and five machine learning algorithms—KNN, random forest, SVM, decision tree, and Gradient Boost—were used to evaluate the performance of the two-stage model. It also demonstrates how the categorization model facilitates achieving acceptable accuracy with any classifier.

Table 5 indicates that the proposed two-stage ensemble model outperformed the other machine learning classifiers, achieving a 99.84% accuracy level, which highlights the effectiveness of the proposed system in identifying individual sentiments about phone products and brands. Additionally, the table shows that SVM and random forest achieved high accuracy, with SVM reaching 94.45% and random forest 93.57% accuracy, suggesting that the new hybrid categorization model simplifies the classification of each tweet. However, the performance of SVM and random forest in identifying the negative category is very low, with precision, sensitivity, and F1-score metrics of 11.13%, 5.22%, and 18.41% for SVM and 13.18%, 6.75%, and 15.15% for random forest, respectively, in the negative category. This indicates that while the categorization method makes it easier to differentiate between positive and neutral opinions, the two-stage proposed model also facilitates distinguishing negative opinions from neutral and positive ones.

Table 5 also shows that the decision tree achieved an acceptable accuracy level of 80.70%. Additionally, the table reveals that KNN and Gradient Boost (an example of an ensemble algorithm) achieved relatively low accuracy, with 68.83% and 64.98%, respectively, indicating lower performance in identifying sentiment about phone brands and products.

To validate the proposed system and ensure that it does not suffer from data overfitting, the system is tested using fourfold and fivefold algorithms. Table 6 shows the results obtained from applying the fourfold and fivefold algorithms to the dataset. Table 6 also demonstrates that five machine learning models—KNN, decision tree, random forest, SVM, and Gradient Boosting—are used to compare their results with those of the proposed system to ensure its performance. Table 6 indicates that the proposed two-stage ensemble classification model outperforms other machine learning classifiers, achieving 99.51% $\pm$ 0.19 and 99.69% $\pm$ 0.18 when the fourfold and fivefold algorithms are applied, respectively. Table 6 also shows that SVM and random forest achieve very high accuracy, indicating that the new hybrid categorization method makes it easier for these algorithms to classify each individual opinion. SVM and random forest achieve 95.87% $\pm$ 5.06 and 92.68% $\pm$ 1.02 with the fourfold algorithm, and 98.56% $\pm$ 2.40 and 92.73% $\pm$ 0.96, respectively, when the fivefold algorithm is applied. Table 6 also shows that the decision tree achieves acceptable accuracy, with 79.16% $\pm$ 1.94 and 79.16% $\pm$ 1.81 for the fourfold and fivefold algorithms, respectively. However, Table 6 also indicates that KNN and gradient boosting are poor choices for classifying individual opinions about phone brands and products, as they achieve very low accuracy.

4.4 Discussion

As shown in Tables 5 and 6, the two-stage proposed method outperformed other machine learning classifiers, achieving 99.84% accuracy and proving its ability to differentiate between the three categories, as seen in the precision, sensitivity, and F1-score for each sentiment category in Tables 5 and 6. The high accuracy results demonstrate the system's capability to analyze tweets about phone products and brands and identify the sentiment of each tweet. Tables 5 and 6 also show that random forest and SVM achieved very high accuracy, indicating that the categorization step was performed well and helped the classifiers differentiate between neutral and positive tweets. However, the precision, sensitivity, and F1-score for negative tweets were lower, suggesting that the two-stage classification model most effectively distinguishes negative tweets from the other two categories (positive and neutral).

The proposed system is also compared with related works to evaluate its performance. Table 7 provides a detailed comparison between the proposed system and related works. Table 7 shows that the proposed system outperforms all related works that used machine learning or deep learning in terms of accuracy, time, and resource consumption. The proposed system uses machine learning algorithms that require minimal time and resources to

Table 7 Comparison between proposed system and related work

Related work	algorithm(s) Used	Optimization algorithm	Dataset	Accuracy	Time and resource consumed
Anjum et al. [22]	Logistic Regression	–	The authors used five datasets: Sentiment Analysis 100,000 tweets [23], 1000 Tweets [24], Amazon Labeled Dataset [25], Yelp Dataset [26], and Amazon Mobiles Dataset [27]	The author's system achieves 93.74% accuracy when applied to 100,000 tweets, 1000 Tweets	Low as it uses a machine learning algorithm
Shaheen et al. [28]	Random forest	–	The authors used a dataset collected from Kaggle, containing 400,000 individual consumer reviews of unlocked phones on the Amazon platform	. The author's system achieves 85% accuracy	Low as it uses a machine learning algorithm
Kayakuş et al. [29]	SVM	–	The authors tested their system on a dataset containing consumer reviews of the iPhone 11, collected from Trendyol	The author's system achieves 50% accuracy	Low as it uses a machine learning algorithm
Hossain and Rahman [30]	Logistic Regression	–	The authors collected a dataset from the Yelp website	The author's system achieves 93.77% accuracy	Low as it uses a machine learning algorithm
Ma'ruf et al. [31]	Ensemble algorithm, which is a combination of KNN, SVM, and Naïve Bayes		The authors collected a dataset consisting of 510 consumer reviews about smartphones from Shopee and Tokopedia platforms	The author's system achieves 89.22% accuracy	Low as it uses a machine learning algorithm
Lagrari and Elkettani [32]	Modified version of BERT model	LA metaheuristic algorithm	The authors use Brands and Product Emotions dataset	The author's system achieves 92.2% accuracy	It consumes many resources and times as it uses deep learning algorithms
Iqbal et al. [34]	LSTM	–	The authors used five datasets: Yelp, Amazon Fine Food Reviews, Cell Phones and Accessories, Amazon Products, and IMDB	The author's system achieves 97% accuracy when it applied on Accessories, Amazon Products dataset	It consumes many resources and times as it uses deep learning algorithms
Kaur and Sharma [35]	LSTM	–	The authors used three different datasets: SemEval-2014, Sentiment140, and STS-Gold	The author's system achieves 95.9% accuracy	It consumes many resources and times as it uses deep learning algorithms
Alzahrani et al. [36]	Hybrid model combining CNN and LSTM algorithms	–	The authors collected a dataset containing 9,500 individual reviews about phone, tablet, television, and laptop brands and products from the Amazon website	The author's system achieves 94% accuracy	It consumes many resources and times as it uses deep learning algorithms
Rasappan et al. [37]	LSTM	EGJO and IGWO	The authors used a dataset collected from Prompt Cloud	The author's system achieves 97.65% accuracy	It consumes many resources and times as it uses deep learning algorithms
Proposed system	Propose a new two-stage ensemble algorithm that combines KNN, Random Forest, and Neural Network	Proposed a new modified version of the Walrus optimization algorithm	Brands and Product Emotions dataset	The proposed system achieves 99.84% accuracy	Low as it uses a machine learning algorithm

classify each tweet, and the proposed categorization method reduces the dataset's dimensionality, minimizing the classification time for each tweet. Additionally, the proposed system successfully achieved 99.84% accuracy in classifying individuals' opinions about phone products and brands into neutral, positive, and negative categories.

Table 7 also shows that related works using machine learning algorithms to identify individual opinions about phone products achieved accuracy ranging from 50 to 93.77%, which is not considered high and needs improvement to increase accuracy, although it requires less time to classify each tweet. Table 7 further shows that related works using deep learning techniques achieved accuracy ranging from 92.2 to 97.65%, which is considered very high, but these methods consume more resources and time than the proposed system. Notably, only Lagrari and Elkettani [32] used the same dataset as this paper, achieving a 92.2% accuracy level using a modified version of the BERT algorithm, a deep learning method that consumes many resources and time. In contrast, the proposed system achieved 99.84% accuracy using machine learning techniques that consume fewer resources and time.

5 Conclusion and future work

This paper introduces a new sentiment analysis system capable of classifying individual opinions about phone brands and products as negative, positive, or neutral using machine learning techniques and the Twitter platform. It also presents a new green hybrid categorization method that transforms text data into a numerical representation, making it easier to classify. This categorization method relies on a combination of the bag-of-words, TF-IDF, and SVD algorithms. Additionally, a modified version of the Walrus Optimization Algorithm, based on orthogonal learning techniques, is proposed to address the slow convergence and risk of overfitting issues found in the original Walrus Optimization Algorithm when identifying optimal hyperparameter values for machine learning algorithms. This modified Walrus Optimization Algorithm is used to determine the optimal hyperparameter values for the proposed ensemble algorithm. Furthermore, a new machine learning-based two-stage ensemble algorithm is proposed to classify tweets as neutral, positive, or negative opinions about phone products and brands. This ensemble algorithm combines three machine learning algorithms: KNN, random forest, and neural network. The algorithm operates in two stages: in the first stage, the results from the categorization model are trained and tested individually using KNN and random forest, and then the results of these two algorithms are combined and further trained and tested using a neural network in the second stage to make the final decision about each tweet. The proposed system outperforms deep and machine learning classifiers and related works regarding accuracy and response time, achieving a 99.84% accuracy rate when applied to the Brands and Product Emotions dataset. Future work will focus on adapting the system to other domains and languages to generalize its utility further. Future research could build upon the proposed system, such as expanding the system to handle sentiments in multiple languages using advanced multilingual NLP techniques, which can increase its global applicability. Developing mechanisms for real-time sentiment monitoring using distributed computing frameworks can enhance scalability and timeliness. Enhanced Contextual Understanding: Integrating context-aware models like BERT can improve handling sarcasm, irony, and ambiguous sentiments. Addressing data privacy and algorithmic biases is crucial for moral and fair sentiment analysis.

Author contributions This paper presents several contributions to sentiment analysis, such as: Designed and implemented a new sentiment analysis tool for phone products and brands on the Twitter platform that classifies each tweet as a positive, negative, or neutral opinion about the brand or product. Proposed a new hybrid categorization tool that categorizes each word in the tweets and calculates the frequency of each word in the dataset based on a combination of Bag of Words, Term Frequency-Inverse Document Frequency (TF-IDF), and Singular Value Decomposition (SVD) algorithms. Proposed a new ensemble classification model that combines K-Nearest Neighbors (KNN), Random Forest, and neural networks. Developed a modified version of the Walrus optimizer to identify the optimal hyperparameter values for the KNN, random forest, and neural network models.

Funding Open access funding provided by The Science, Technology & Innovation Funding Authority (STDF) in cooperation with The Egyptian Knowledge Bank (EKB). Open access agreement for Egypt.

Data availability This dataset Available in https://data.world/crowdflower/brands-and-product-emotions.

References

1. Kim Y, Dwivedi R, Zhang J, Jeong SR (2016) Competitive intelligence in social media Twitter: iPhone 6 vs. Galaxy S5. Online Inf Rev 40(1):42–61
2. Arora D, Li KF, Neville SW (2015) 'Consumers' sentiment analysis of popular phone brands and operating system preference using Twitter data: a feasibility study'. In: 2015 IEEE 29th international conference on advanced information networking and applications, pp 680–686
3. Sharma SK, Daga M, Gemini B (2020) 'Twitter sentiment analysis for brand reputation of smart phone companies in India'. In Proceedings of ICETIT 2019: emerging trends in information technology, pp 841–852
4. Rane N (2023) 'Enhancing customer loyalty through Artificial Intelligence (AI), Internet of Things (IoT), and Big Data technologies: improving customer satisfaction, engagement, relationship, and experience', Internet of Things (IoT), and Big Data Technologies: Improving Customer Satisfaction, Engagement, Relationship, and Experience (October 13, 2023)
5. Alslaity A, Orji R (2024) Machine learning techniques for emotion detection and sentiment analysis: current state, challenges, and future directions. Behav Inf Technol 43(1):139–164
6. Tan KL, Lee CP, Lim KM (2023) A survey of sentiment analysis: Approaches, datasets, and future research. Appl Sci 13(7):4550
7. Rodríguez-Ibánez M, Casánez-Ventura A, Castejón-Mateos F, Cuenca-Jiménez P-M (2023) A review on sentiment analysis from social media platforms. Expert Syst Appl 223:119862
8. Rust RT, Rand W, Huang M-H, Stephen AT, Brooks G, Chabuk T (2021) Real-time brand reputation tracking using social media. J Mark 85(4):21–43
9. "Twitter by the numbers (2023): Stats, Demographics & Fun Facts," Omnicore Agency, 12-Jan-2024. [Online]. Available: https://www.omnicoreagency.com/twitter-statistics/. Accessed 21 Aug 2024
10. Abualigah L, Kareem NK, Omari M, Elaziz MA, Gandomi AH (2021) 'Survey on Twitter sentiment analysis: Architecture, classifications, and challenges', Deep learning approaches for spoken and natural language processing, pp 1–18
11. Weber CT, Syed S (2019) 'Interdisciplinary optimism? Sentiment analysis of Twitter data.' Royal Soc Open Sci 6(7):190473
12. Qi Y, Shabrina Z (2023) Sentiment analysis using Twitter data: a comparative application of lexicon-and machine-learning-based approach. Soc Netw Anal Min 13(1):31
13. Antonakaki D, Fragopoulou P, Ioannidis S (2021) A survey of Twitter research: Data model, graph structure, sentiment analysis and attacks. Expert Syst Appl 164:114006
14. Shankar S, Tewari V (2021) Understanding the emotional intelligence discourse on social media: insights from the analysis of twitter. J Intell 9(04):56
15. Wang Y, Guo J, Yuan C, Li B (2022) Sentiment analysis of Twitter data. Appl Sci 12(22):11775
16. Shiha M, Ayvaz S (2017) The effects of emoji in sentiment analysis. Int J Comput Electr Eng (IJCEE) 9(1):360–369
17. Nazman NNN, Ting SH, Chuah KM (2021) 'Social Media Users' Reasons for Using Non-Standard Words in Twitter', Editorial Committee, 103
18. Giachanou A, Crestani F (2016) Like it or not: a survey of twitter sentiment analysis methods. ACM Comput Surv (CSUR) 49(2):1–41
19. Raghunathan N, Saravanakumar K (2023) Challenges and issues in sentiment analysis: a comprehensive survey. IEEE Access 11:69626–69642
20. Rajadesingan A, Zafarani R, Liu H (2015) 'Sarcasm detection on twitter: a behavioral modeling approach'. In: Proceedings of the eighth ACM international conference on web search and data mining, pp 97–106
21. Mihi S, Ait Ben Ali B, Laachfoubi N (2022) A comparative review of Tweets Automatic Sarcasm Detection in Arabic and English'. In: International conference on advanced intelligent systems for sustainable development, pp 841–849

22. Anjum M, Hussain S, Kazmi R, Bajwa IS, Ali MA (2021) Sentiment analysis of Twitter tweets for mobile phone brands. Pak J Eng Technol 4(1):131–138
23. Shyamasundar LB, Rani PJ (2016) 'Twitter sentiment analysis with different feature extractors and dimensionality reduction using supervised learning algorithms'. In: 2016 IEEE Annual India Conference (INDICON), pp 1–6
24. Kumari U, Sharma AK, Soni D (2017) 'Sentiment analysis of smart phone product review using SVM classification technique'. In: 2017 International conference on energy, communication, data analytics and soft computing (ICECDS), pp 1469–1474
25. Zhou X, Tao X, Yong J, Yang Z (2013) 'Sentiment analysis on tweets for social events'. In: Proceedings of the 2013 IEEE 17th international conference on computer supported cooperative work in design (CSCWD), pp 557–562
26. Rathi M, Malik A, Varshney D, Sharma R, Mendiratta S (2018) 'Sentiment analysis of tweets using machine learning approach'. In: 2018 11th International conference on contemporary computing (IC3), pp 1–3
27. Raut VB, Londhe DD (2014) Opinion mining and summarization of hotel reviews. In: 2014 International conference on computational intelligence and communication networks, pp 556–559
28. Shaheen M, Awan SM, Hussain N, Gondal ZA (2019) Sentiment analysis on mobile phone reviews using supervised learning techniques. Int J Mod Educ Comput Sci 10(7):32
29. Kayakuş M, Yiğit Açikgöz F, Dinca MN, Kabas O (2024) Sustainable brand reputation: evaluation of iPhone customer reviews with machine learning and sentiment analysis. Sustainability 16(14):6121
30. Hossain MS, Rahman MF (2023) Customer sentiment analysis and prediction of insurance products' reviews using machine learning approaches. FIIB Bus Rev 12(4):386–402
31. Ma'ruf M, Kuncoro AP, Subarkah P, Nida F (2022) Sentiment analysis of customer satisfaction levels on smartphone products using Ensemble Learning. Ilk J Ilm 14(3):339–347
32. Lagrari F-E, Elkettani Y (2020) Customized BERT with convolution model: a new heuristic enabled encoder for Twitter sentiment analysis. Int J Adv Comput Sci Appl 11(10):9
33. "Brands and product emotions – dataset," crowdflower. data.world, 21 Nov 2016. [Online]. Available: https://data.world/crowdflower/brands-and-product-emotions. Accessed 21 Aug 2024
34. Iqbal A, Amin R, Iqbal J, Alroobaea R, Binmahfoudh A, Hussain M (2022) Sentiment analysis of consumer reviews using deep learning. Sustainability 14(17):10844
35. Kaur G, Sharma A (2023) A deep learning-based model using hybrid feature extraction approach for consumer sentiment analysis. J of big data 10(1):5
36. Alzahrani ME, Aldhyani THH, Alsubari SN, Althobaiti MM, Fahad A (2022) Developing an intelligent system with deep learning algorithms for sentiment analysis of e-commerce product reviews. Comput Intell Neurosci 2022(1):3840071
37. Rasappan P, Premkumar M, Sinha G, Chandrasekaran K (2024) Transforming sentiment analysis for e-commerce product reviews: hybrid deep learning model with an innovative term weighting and feature selection. Inf Process Manag 61(3):103654

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Authors and Affiliations

Gomaa Elmokhtar[1] · Waleed M. Mohamed[2] · Engy El-Shafeiy[1] · Eman M. G. Younis[2]

✉ Gomaa Elmokhtar
gomaa.elmokhtar@fcai.usc.edu.eg

Waleed M. Mohamed
waleedmakram@mu.edu.eg

Engy El-Shafeiy
Engy.Elshafeiy@fcai.usc.edu.eg

Eman M. G. Younis
eman.younas@mu.edu.eg

[1] Faculty of Computers and Artificial Intelligence, University of Sadat City, Sadat, Egypt

[2] Faculty of Computers and Information, Minia University, Minya, Egypt